Prêt à dépenser

Implémentez un Modèle de Scoring

Synthèse

Contexte & Objectifs

PROJET

Construction d'un modèle de credit scoring

- prédiction de la probabilité de défaut de l'emprunteur
- accessibilité du modèle aux utilisateurs finaux
- interprétabilité & reproductibilité des résultats

DÉMARCHE

- model registry avec MLFlow
- model explanations avec Shap
- UI via Streamlit Community Cloud…
- … sur la base d'une API Flask
- versioning avec GitHub
- unit tests automatisés avec GitHub Actions

➢ Mise en place d'un pipeline CI / CD

Prêt à dépenser

Sommaire

Prêt à dépenser

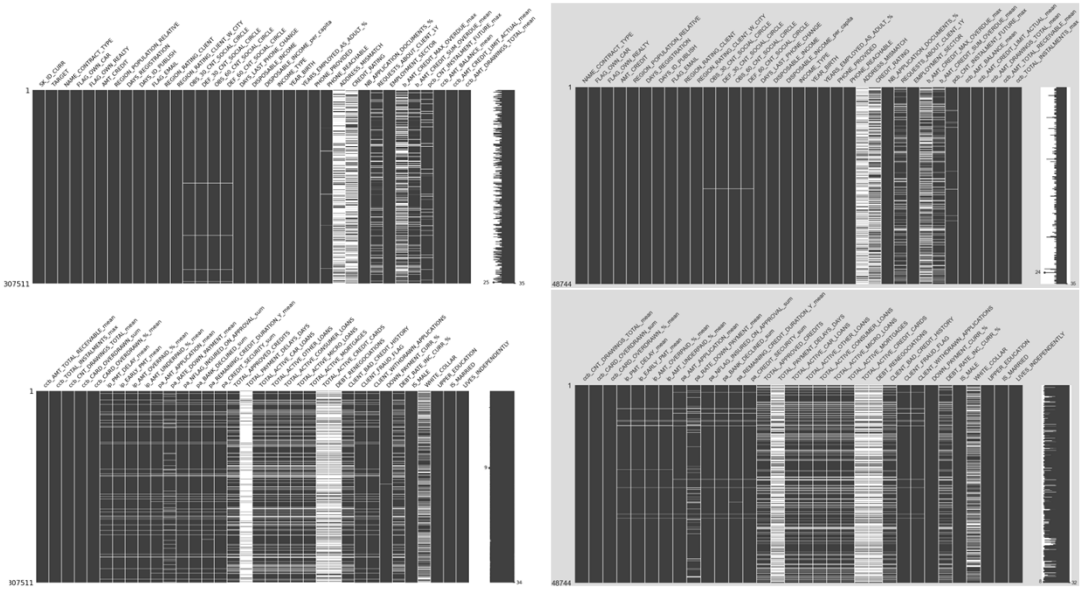
1 – Analyse Exploratoire des Données

1.1 – Nettoyage

- Current applications with Home Credit :
  - application_train.csv (305,511 x 122)
  - application_test.csv (48,744 x 121)
- Previous applications with Home Credit :
  - previous_application.csv (1,670,214 x 37)
  - POS_CASH_balance.csv (10,001,358 x 8)
  - instalments_payments.csv (13,605,401 x 8)
  - credit_card_balance.csv (3,840,312 x 23)
- Previous applications with other credit providers :
  - bureau.csv (1,716,428 x 17)
  - bureau_balance.csv (27,299,925 x 3)

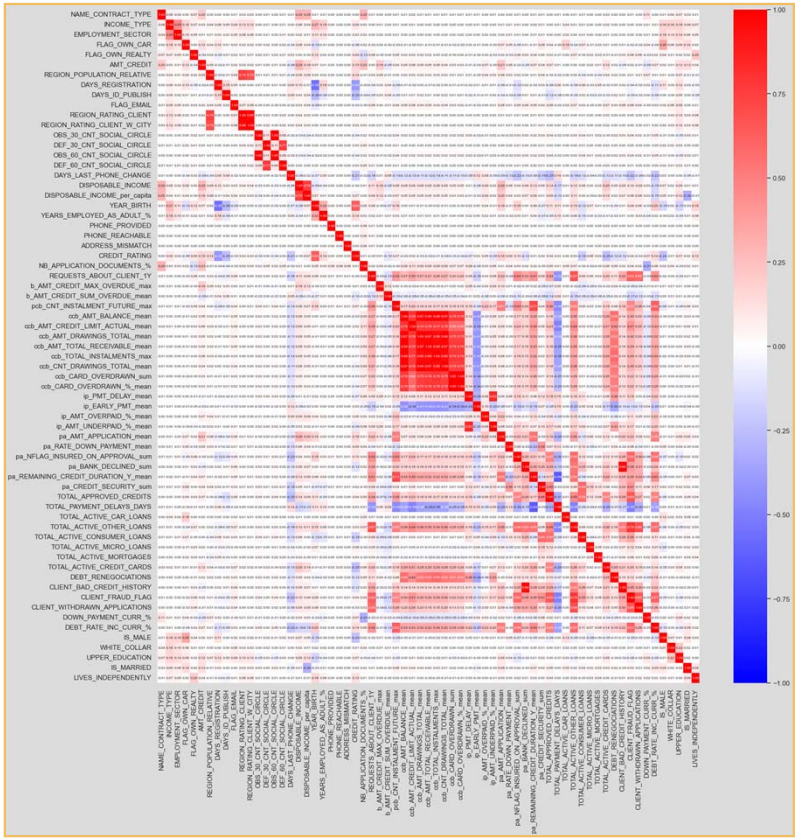
Feature engineering, joins & aggregations

- Training data (307,511 x 69)
- Test data (48,744 x 69)



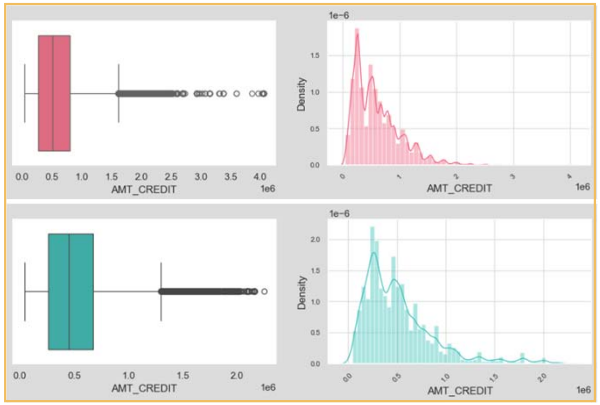
1 – Analyse Exploratoire des Données

1.2 – Exploration



Retraitements :
Outliers,
NaNs imputations &
Correlated features

- Training data (307,511 x 56)
- Test data (48,744 x 56)



- Features ne suivent pas une distribution Gaussienne
- Présence de skew…
- … & de variables excessivement corrélées

Prêt à dépenser

1 – Analyse Exploratoire des Données

1.3 – Feature Engineering

- Données socio-économiques
  - âge
  - genre
  - situation maritale
  - niveau d'éducation
  - secteur d'activité
  - revenus par tête du foyer…
- Indicateurs de fraude potentielle
  - email/numéro de téléphone joignable
  - récence des coordonnées de contact
  - suspicion de fraude passée…
- Indicateurs de risque de crédit
  - credit score
  - présence d'une assurance sur les crédits existants
  - encours & mensualités totaux des crédits existants
  - taux d'endettement
  - montant du collatéral
  - nombre & montant des défauts de paiement passés…

56 FEATURES*

- 3 variables catégorielles
- 11 variables booléennes
- 21 variables numériques discrètes
- 22 variables numériques continues

1 TARGET

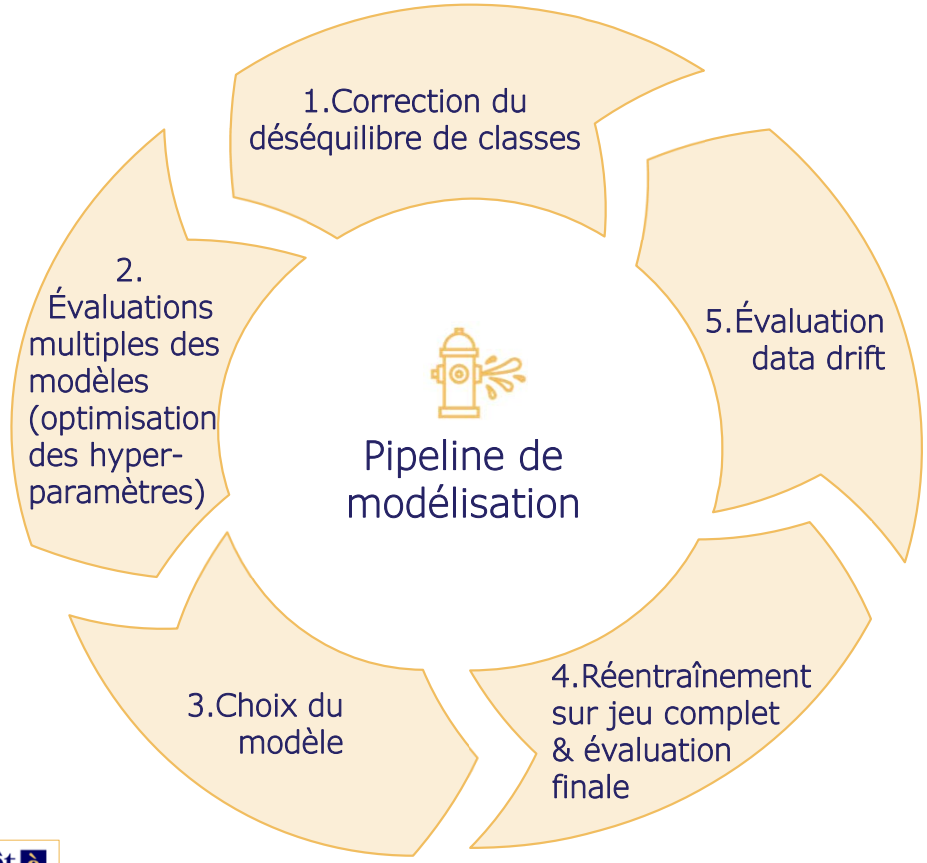
- Défaut de remboursement du crédit
  - 8.07% des clients du jeu d'entraînement
  - Classes Déséquilibrées

Prêt à dépenser

* Voir fichier data_dictionary.xlsx du repo https://github.com/CelineBoutinon/credit-scoring.git/references pour la liste complète, les sources & les modalités de calcul de chaque variable

2 – Modélisation

2.1 – Méthodologie



1. Techniques Utilisées:

- Stratified sampling train/val/test
- SMOTE over-sampling
- Random under-sampling

2. Métriques d'Évaluation:

- F1 score
- F2 score
- AUC
- Accuracy / balanced accuracy
- Precision / average precision
- Score métier

3. Modèles Testés:

- Régression Logistique (baseline)
- LightGBM
- XGBoost

Prêt à dépenser

2 – Modélisation

2.2 – Score Métier

- Objectif : maximiser le rendement en pénalisant les erreurs de prédiction (faux négatifs et faux positifs) en fonction de leur coût réel et de leur coût d'opportunité respectifs pour la banque:
  - Vrais positifs = 0 (prêt refusé => aucun revenu & aucun risque de perte)
  - Vrais négatifs = 0,05 (prêt accordé => gain = marge moyenne 5%)
  - Faux positifs = -0.05 (prêt refusé à tort => coût d'opportunité = marge moyenne 5% perdue)
  - Faux négatifs = -0.5* (prêt accordé à tort => coût réel 10 fois supérieur à celui des faux positifs)

- Fonction de gain : à maximiser

$$Score\ Métier = 0 * TP + 0.05 * TN - 0.05 * FP - 0.5 * FN$$

Prêt à dépenser

* Coût total estimé suite au défaut du client et après prise en compte des intérêts non-perçus sur la durée résiduelle du prêt au moment du défaut et de la possibilité de recouvrement partiel par saisie et cession des actifs garantis & autres biens mobiliers et/ou immobiliers du client.

2 – Modélisation

2.3 – Choix du Modèle

➢ Évaluation sur le jeu de validation :

	Validation Metrics										
Model	F1 Score	F2 Score	AUC	Accuracy	Balanced Accuracy	Precision	Average Precision	Recall	Best k	Score Metier	Optimal Threshold
Regression Logistique	0.2486	0.3969	0.6696	0.6784	0.6696	0.1532	0.1285	0.6590	11	0.6551	0.4690
LightGBM	0.2627	0.4102	0.6814	0.7028	0.6814	0.1642	0.1355	0.6557	13	0.6809	0.4770
XGBoost	0.2311	0.3887	0.6607	0.6169	0.6607	0.1379	0.1215	0.7129	3	0.6696	0.5780

➢ Réévaluation sur le jeu de test :

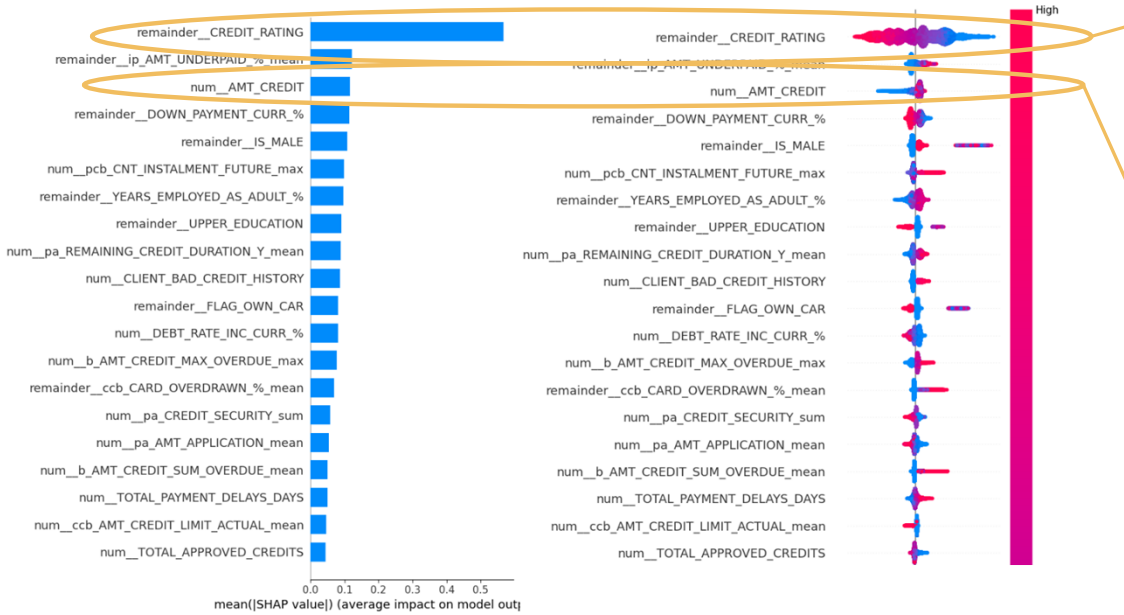
	Test Metrics										
	F1 Score	F2 Score	AUC	Accuracy	Balanced Accuracy	Precision	Average Precision	Recall	Best k	Score Metier	Optimal Threshold
LightGBM	0.2632	0.4109	0.6819	0.7034	0.6819	0.1646	0.1358	0.6563	13	0.6927	0.502

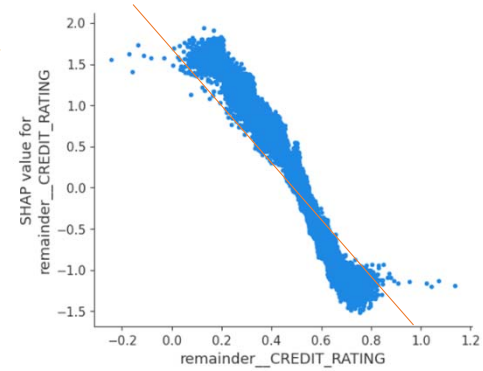
Prêt à dépenser

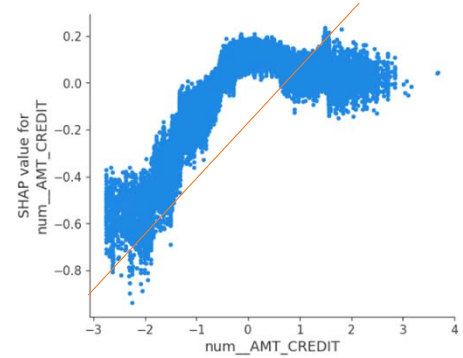
2 – Modélisation

2.3 – Choix du Modèle

➢ Feature importance globale*:







* Pour une analyse de l'importance locale des features, se référer aux prédictions de l'application Streamlit sur les dossiers clients individuels (voir exemple au point 3.3 slide 15).

3 – Mise en Production

3.1 – Data Drift

- Whole dataset:

Dataset Drift
Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5

56	10	0.179
Columns	Drifted Columns	Share of Drifted Columns

Data Drift Summary

Drift is detected for 17.857% of columns (10 out of 56).

Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
AMT_CREDIT	num			Detected	Wasserstein distance (normed)	0.207334
CLIENT_WITHDRAWN_APPLICATIONS	num			Detected	Wasserstein distance (normed)	0.193724

- Positive Class:

Dataset Drift
Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5

56	13	0.232
Columns	Drifted Columns	Share of Drifted Columns

Data Drift Summary

Drift is detected for 23.214% of columns (13 out of 56).

Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
AMT_CREDIT	num			Detected	Wasserstein distance (normed)	0.216557
CLIENT_WITHDRAWN_APPLICATIONS	num			Detected	Wasserstein distance (normed)	0.207555

- Negative class:

Dataset Drift
Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5

56	14	0.25
Columns	Drifted Columns	Share of Drifted Columns

Data Drift Summary

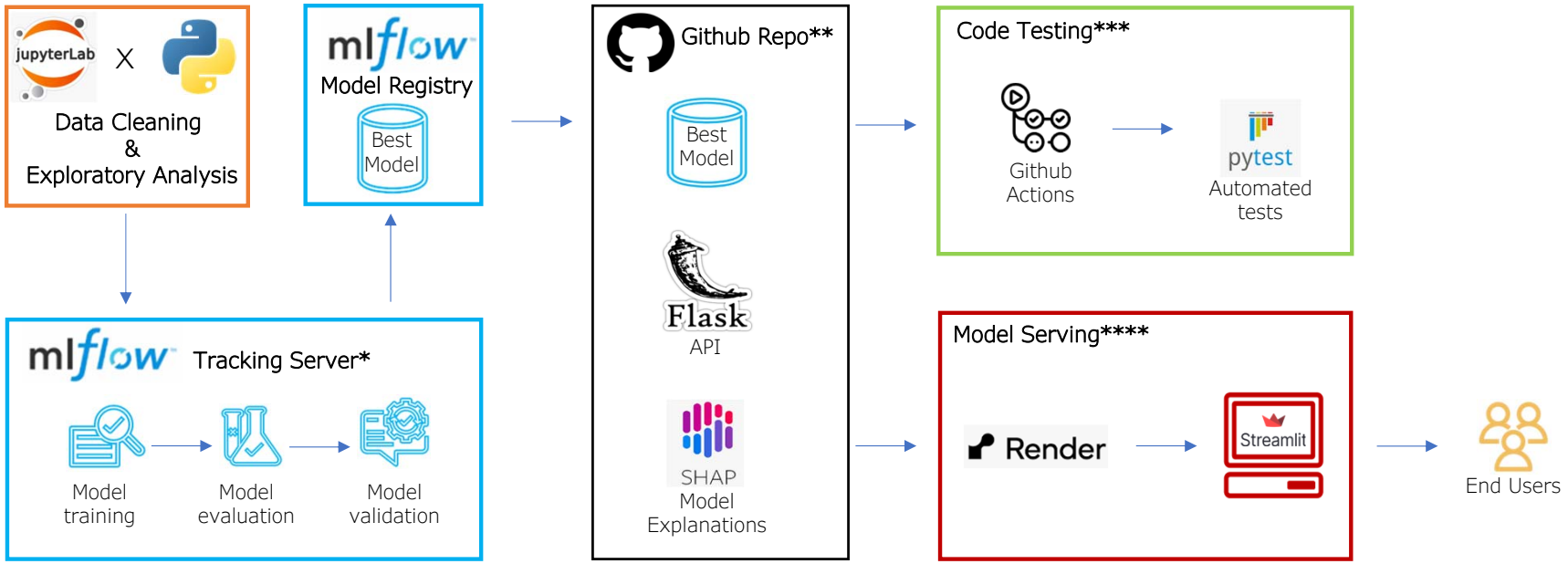
Drift is detected for 25.0% of columns (14 out of 56).

Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
AMT_CREDIT	num			Detected	Wasserstein distance (normed)	0.210758
TOTAL_PAYMENT_DELAYS_DAYS	num			Detected	Wasserstein distance (normed)	0.208908

Prêt à dépenser

3 – Mise en Production

3.2 – Déploiement Continu (1/2)



* Voir Annexe 1 ** Voir Annexe 2 *** Voir Annexe 3 **** Voir Annexes 4 & 5

3 – Mise en Production

3.2 – Déploiement Continu (2/2)

- GitHub repos :
  - Analyse exploratoire & modélisation
    https://github.com/CelineBoutinon/credit-scoring.git
  - API, unit tests & application Streamlit
    https://github.com/CelineBoutinon/credit-scoring-api.git

- API disponible sur :
  https://credit-scoring-api-0p1u.onrender.com

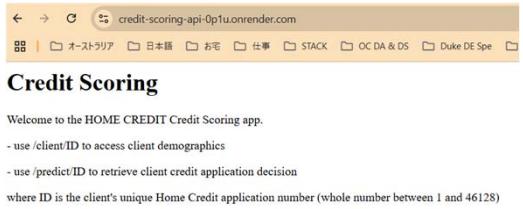
- Streamlit App disponible sur :
  https://credit-scoring-api-f6g7yxlaqnt3qhu7rk7rkf.streamlit.app/

Prêt à dépenser

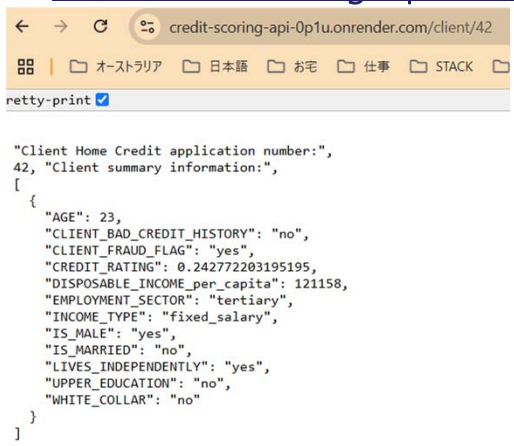
3 – Mise en Production

3.3 – API – Cloud Démo

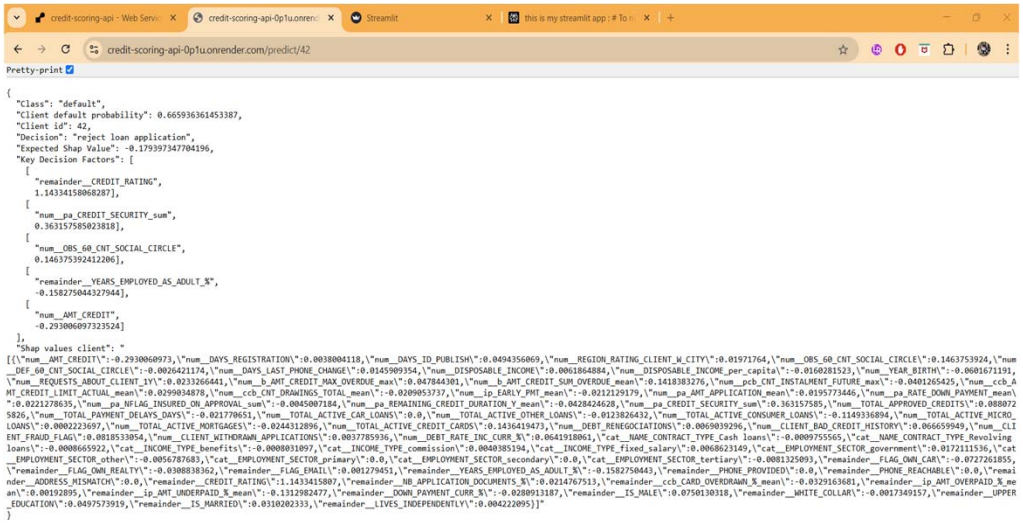
- API landing page :



- API client demographics example :

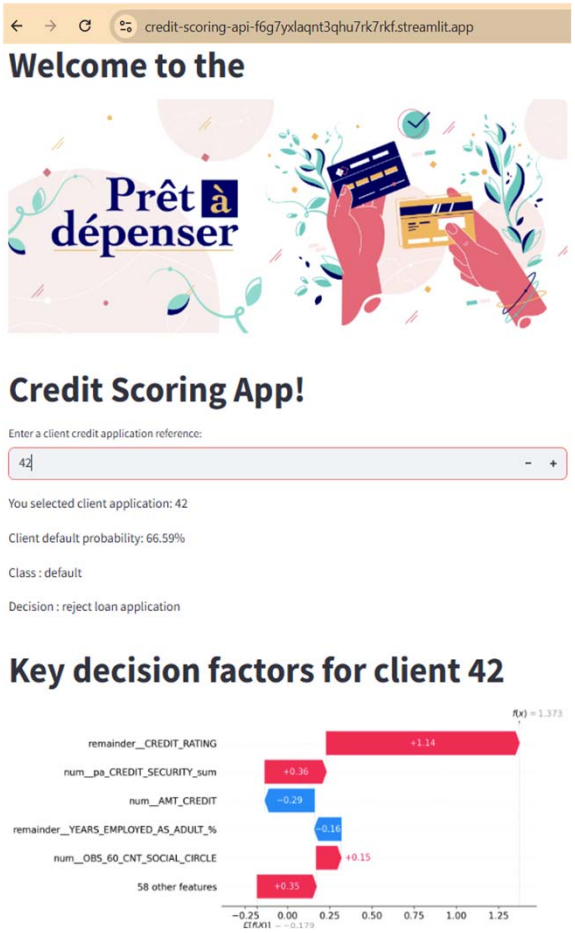


- API prediction example :



3 – Mise en Production

3.3 – Streamlit App – Cloud Démo



Conclusion & perspectives

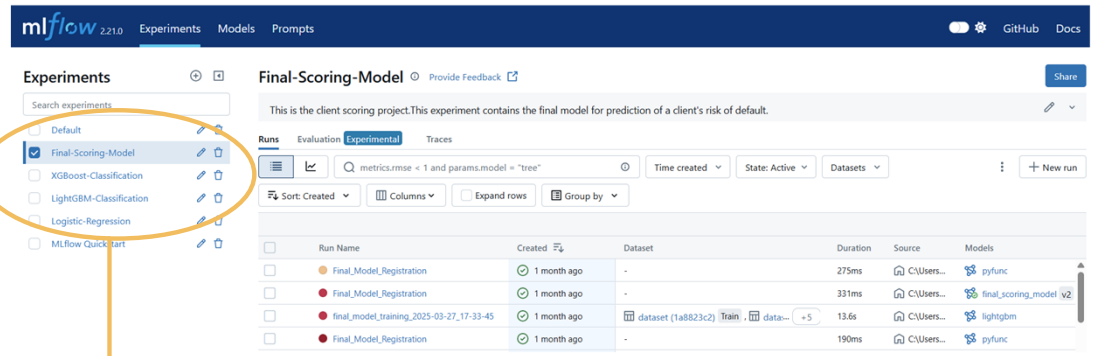
- Déséquilibre de classes nécessite des ajustements pour une modélisation pertinente…
- … et pour le suivi des modèles en production:
  - détection anticipée du data drift
  - nécessité de monitorer aussi le target drift
- Coût des erreurs de prédiction étroitement lié au contexte métier
  - ex. épidémiologie/virologie vs. finance – attention à ne pas minimiser les faux positifs
  - coûts individuels & collectifs réels, pas seulement des coûts d'opportunité

Prêt à dépenser

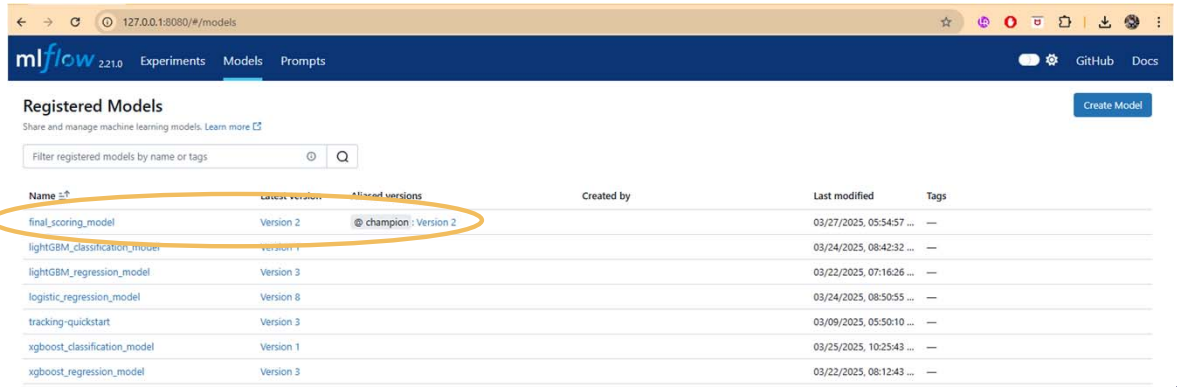
ANNEXES

Annexe 1 - MLFlow Tracking Server

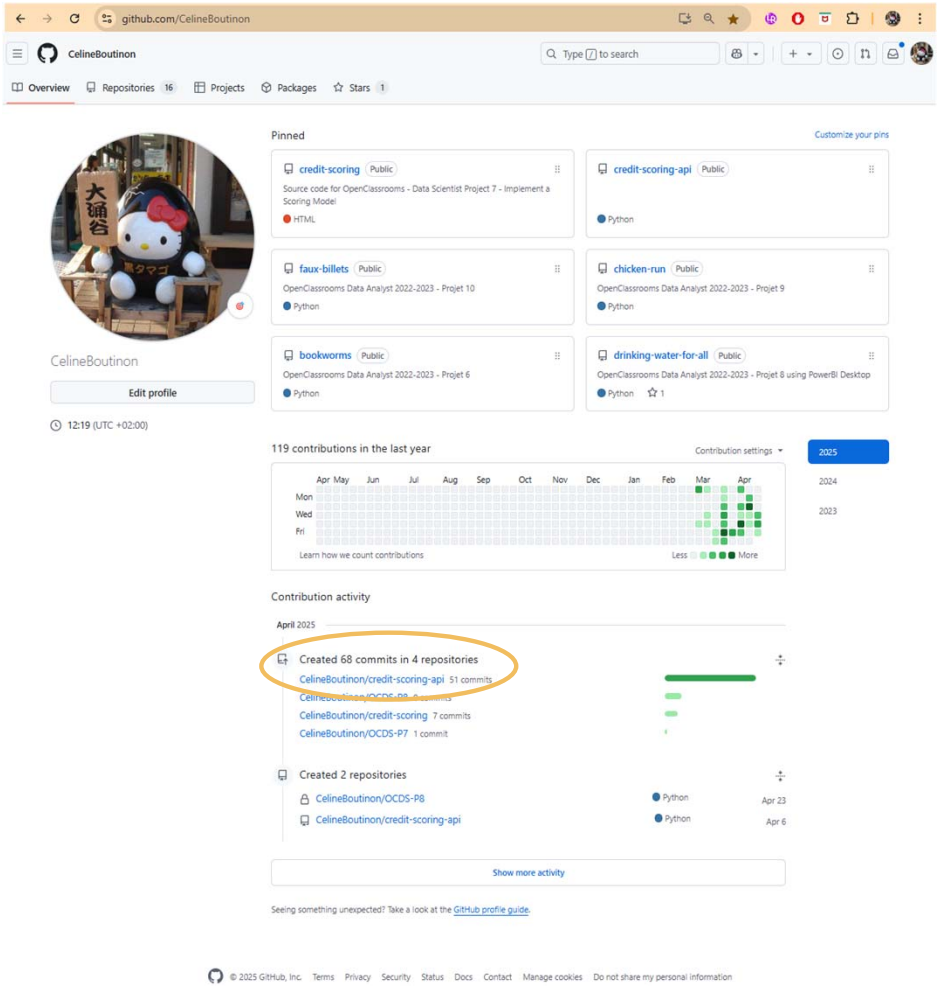
Experiment Tracking :



Model Registry Logging :

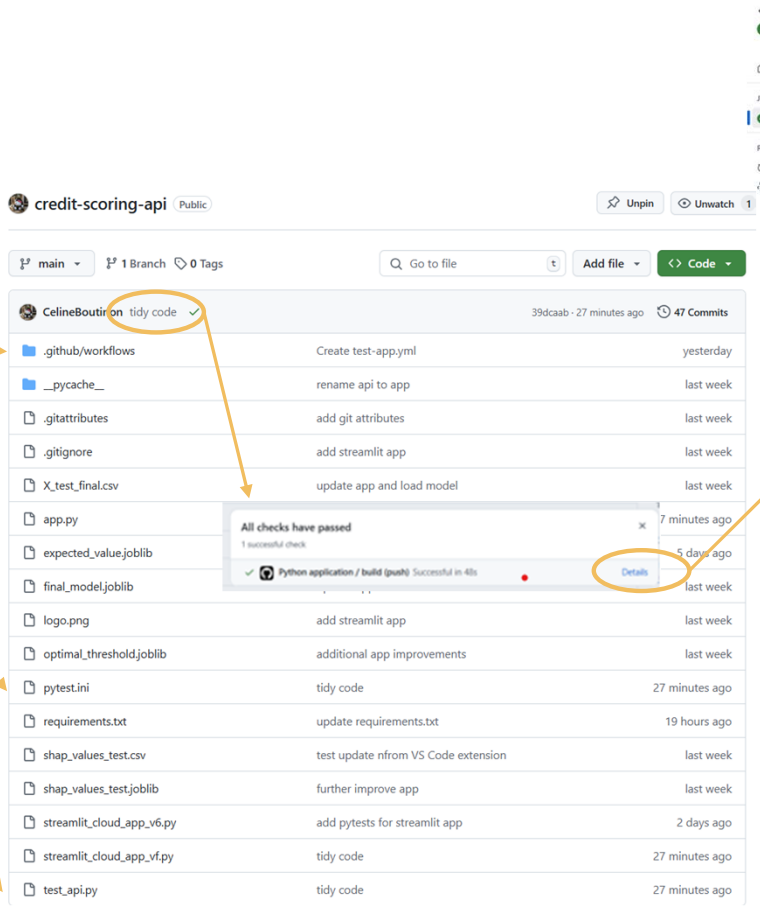


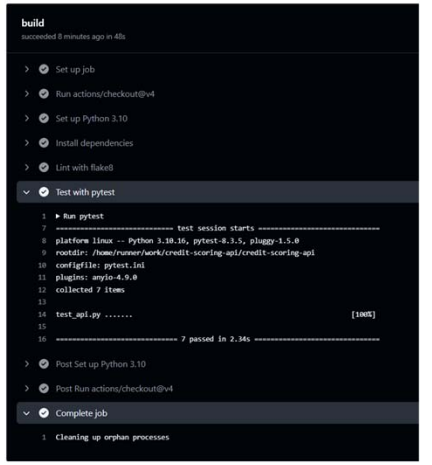
Annexe 2 - Commit history as of 25/04/25



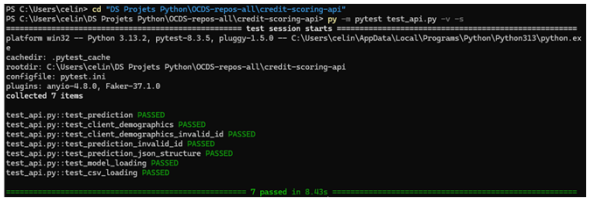
Annexe 3 - Unit Tests implementation with GitHub Actions

The test_api.py file is triggered by a workflow upon each push to the repo according to the config in pytest.ini

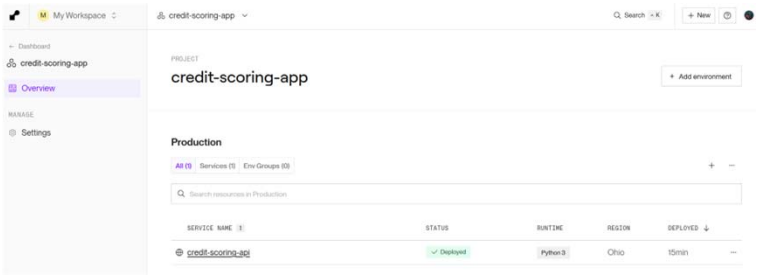




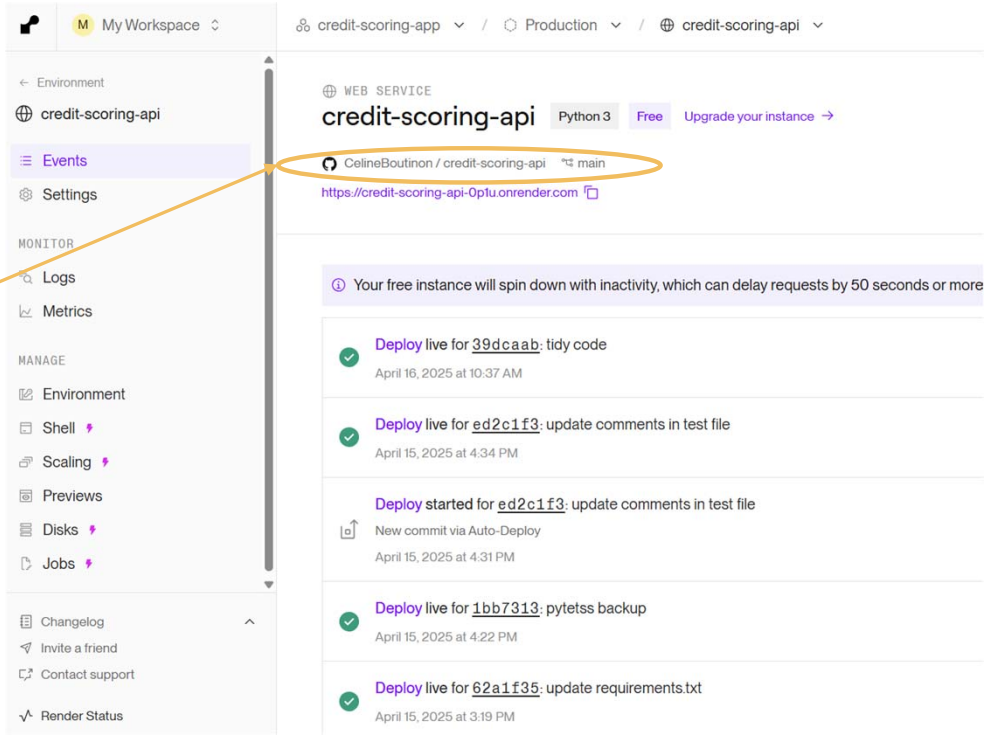
➢ 7 tests successfully passed in accordance with local results :



Annexe 4 - API Cloud deployment with Render.com



The API auto-deploys to Render.com upon each push to GitHub Repo credit-scoring-api



Annexe 5 - App Cloud deployment with Streamlit Community Cloud

The app auto-refreshes upon each push to GitHub Repo credit-scoring-api